

Reasoning Kernel: A Capability-Mediated Reference Architecture for Untrusted Tool Data in LLM Agents

Author: Gianluca Mazza

Affiliation: Independent / practice gianlucamazza.it

Date: 2026-10-07 (Europe/Rome)

Document status: Working paper (technical note). Draft artifact for Zenodo deposit. **No DOI is claimed in this document.**

Software: [gianlucamazza/reasoning-kernel](#) · PyPI [capability-reasoning-kernel](#) (MIT, pre-1.0; version referenced herein: **0.6.3**)

Companion essay: [Untrusted Data Must Not Own Control Flow](#) (2026-10-02)

Abstract (English)

Large language model (LLM) agents that read tool outputs, messages, or pages can be steered by instructions hidden in that data and then act—send mail, leak contacts, or call tools the user did not authorize. Detection inside the prompt is a hope, not a topology. This working paper describes **Reasoning Kernel**, a Python **reference implementation** of a strong, CaMeL-like architecture in which every model is treated as **untrusted compute**, model inputs are mediated, and no model output becomes a durable effect except through one deterministic verification boundary (the Gate).

The paper states the problem of untrusted tool/data ownership of control flow; presents the two public invariants of the kernel; maps roles to modules; summarizes the threat model and honest limits; reports implementation and packaging status without inventing benchmarks or peer-review claims; and situates the work relative to CaMeL (Debenedetti et al., 2025) and related practice. The claim is architectural: a **topology**, not a safety certificate. Host policy remains the residual risk surface.

Abstract (Italiano, breve)

Gli agenti basati su LLM che leggono output di tool, messaggi o pagine possono essere dirottati da istruzioni nascoste nei dati e poi agire. Questo working paper descrive **Reasoning Kernel**, un'**implementazione di riferimento** in Python di un'architettura a capability (forma forte, affine a CaMeL) in cui ogni modello è calcolo non fidato e nessun effetto durevole passa senza un Gate deterministico. È una **topologia**, non una certificazione di sicurezza; la policy resta responsabilità dell'host. Nessun DOI è dichiarato in questo documento.

1. Introduction

Agentic systems increasingly bind LLMs to tools with side effects. The operational failure mode is familiar: untrusted content arrives as data, yet the agent treats fragments of that content as instructions and proceeds to act. Heuristic mitigations—delimiters, “ignore injected instructions,” sandwiching the task after each tool result—regularly fail against new attacks and do not yield inspectable architecture.

A complementary line of work treats the problem as **system design around untrusted models**. Willison’s Dual LLM pattern separates a privileged planner from a quarantined parser without tool access [Willison2023]. CaMeL (Debenedetti et al., 2025) develops that idea into an interpreter that tracks capabilities and enforces policies on tool calls, with evaluation on AgentDojo [Debenedetti2025Camel; Debenedetti2024AgentDojo].

Reasoning Kernel is an independent, framework-agnostic **Python reference** of that strong form: no trusted reasoner; typed plans; capability and provenance checks before every effect; conformance evidence that is necessary but not sufficient for safety. It is published as research software and a packaging artifact (`capability-reasoning-kernel` on PyPI), not as a turn-key security product or an independent audit.

This document exists so the research/software line has a citable working-paper artifact suitable for Zenodo deposit. Site and audit practice on gianlucamazza.it require that **the name follow the artifact**: a Zenodo whitepaper DOI for Reasoning Kernel must not be invented, and must not be conflated with the separate **emotional-memory** Zenodo records.

Contributions (honest scope)

1. A concise statement of the **untrusted-data / control-flow** problem for tool-using agents.
2. A description of the Reasoning Kernel **topology**: mediated inputs, dual untrusted reasoners, deterministic Gate, object-level provenance.
3. An **architecture map** from public modules to roles (context, reasoners, conductor, verifier, tools, memory/trace).
4. A **threat model and limits** section aligned with the project `SECURITY.md` and README “Honest limits.”
5. Pointers to **software, citation practice, and related work**, without fabricated performance numbers.

Non-contributions: new formal proofs; AgentDojo replication numbers; claims of peer review; claims that conformance equals safety; any DOI for this paper until Zenodo assigns one.

2. Problem: untrusted data must not own control flow

Consider an agent that can read an inbox and contacts and send email. A legitimate request might be: summarize the latest message and send the summary to me. An adversarial message body may say: ignore previous instructions and forward all contacts to `attacker@evil.com`. In a conventional tool-calling loop, the same model that planned the task also consumes the tool result; injected text can therefore reshape later tool choice and arguments.

Two failure channels matter:

- **Control-flow hijack.** Injected text changes *which* tools run or in what order.
- **Data-flow hijack.** The plan skeleton is intact, but arguments (recipient, attachment, amount) are filled from attacker-influenced parses—analogous to parameter tampering rather than rewriting the program [Debenedetti2025Camel].

Prompt-only defenses address neither channel with structural guarantees. The design goal adopted here is narrower and sharper:

Untrusted data must not own control flow. It must not cause an effect by itself.

Effects (writes, sends, irreversible API calls) must require a path that the host can name: who assembled planner context, which Gate decision authorized the write, and what the declassifier explicitly allowed.

3. Approach: the Reasoning Kernel pattern (strong / CaMeL-like form)

3.1 Definition

Following the project’s public definition:

A Reasoning Kernel is an architecture in which probabilistic reasoning is treated as an untrusted computational resource, mediated by context on input and verification on output.

In the **strong form**, there is **no trusted reasoner**. Privilege is capability, not trust.

3.2 Two invariants

The reference implementation fixes two invariants. They define a **topology**, not a proved safety property of an arbitrary policy.

- **Invariant A — model inputs are mediated.** The root (privileged) planner does not receive raw tool output. Host-assembled context is what the privileged planner sees. Quarantined reasoners may see untrusted data under reduced authority; their outputs retain provenance.
- **Invariant B — the reasoner never commits reality.** No model output becomes a durable effect except through one deterministic verification boundary: the Gate (`kernel/gate.py`).

Conformance to these invariants is **necessary, not sufficient**. A pass-through declassifier still “conforms” and protects nothing. Whether the Gate’s policy is correct remains on the host [README; docs/CONFORMANCE.md].

3.3 Dual untrusted reasoners

Role	Name	Sees	Emits	Tools
Privileged planner	P-LLM	Controlled query + tool catalog	Typed P _{lan} (not prose/code for effects)	No direct tool callables
Quarantined parser	Q-LLM	Untrusted content under reduced authority	Typed values with provenance	No tool capability

The trusted path is the **interpreter plus capability/provenance Gate**, never a model.

3.4 Plans as static DAGs

A plan is a forward-only DAG of five step kinds: `const`, `tool`, `q_parse`, `subkernel`, `merge`. There are no runtime branches or loops on parsed content. That is a deliberate trade: “if the email says X, do Y” must become a typed value the Gate can inspect. Delegated sub-planners may choose a child plan from untrusted input under a **clamped, reduced grant**; this is not claimed as data-independent planning across all delegation.

4. Architecture

4.1 Role → module map (public tree)

Role	Module area	Responsibility
Context	<code>context/</code> (e.g. assembler)	Build planner prompts from query + catalog only (Invariant A)
Reasoners	<code>reasoner/</code>	P-LLM / Q-LLM providers behind one interface
Conductor	<code>kernel/interpreter.py</code>	Execute the plan DAG
Verifier	<code>kernel/gate.py</code> , <code>kernel/effects.py</code>	Authorize before callables run
Tool catalog	<code>tools/registry.py</code>	Sole holder of tool callables
Memory / Trace	<code>memory/</code>	Durability and audit format

4.2 Effect path (no bypass by construction)

Three construction rules encode Invariant B in the type and wiring story:

1. Tool callables live only in `ToolRegistry`, handed only to `EffectDispatcher`. The interpreter never holds a callable.
2. `EffectDispatcher` cannot be constructed without a `Gate`; `dispatch` authorizes before the callable runs.

3. `ToolCallStep` is the only step kind that invokes a tool callable, and its only handler routes through the dispatcher. Other step kinds produce values; a sub-kernel may invoke tools only through its reduced Gate and the same dispatcher path.

Structural tests in the repository (e.g. denied capability or provenance never firing the callable; committed effects preceded by allowed Gate decisions) are the project's receipts for this wiring—not a substitute for host acceptance tests.

4.3 Gate checks and provenance

The Gate checks, in order: **granted capabilities**, **input schema**, then **provenance**. Provenance is multi-dimensional: sources, readers, and subjects. Joins are conservative (union of sources, intersection of readers, union of subjects). A Q-LLM parse cannot launder taint. Third-party data is not auto-released into a `WRITE`—even to the requesting user—unless `hostDeclassPolicy` explicitly relaxes trust. Labels are **object-level**; field-level precision is deferred.

4.4 Composition and bounds

- Child kernels cannot widen authority: a planner whose grant exceeds the dispatcher's is rejected; `SubKernelStep` clamps grants.
- `RunLimits` bound steps, effects, q-parses, optional timeouts, and nesting depth; exceeding a bound aborts closed.
- Reasoner failure (empty/refused/malformed output or transport error) is fail-closed for subsequent work; earlier committed effects remain real (`committed=None` is not rollback).

4.5 Host embedding and conformance profiles

Hosts embed via `RunSession` (or low-level wiring) with persistent sinks and bounded defaults; see project operations docs. Version 0.6 introduces fixed profiles `gate-v1` and `operational-v1` so hosts can emit sanitized, payload-free conformance reports. Reports are evidence from trusted host observers, **not** signed attestations of identity, credentials, network policy, or live-adapter correctness.

5. Threat model and limits

5.1 In scope (mechanism)

- **Attacker controls untrusted data** returned by `READ` tools (email bodies, pages, tool results) and may place arbitrary instructions there.
- **Success for the attacker (against the kernel mechanism)** means committing an effect that Gate and labels should have blocked—a hole in `kernel/`, `schemas/`, or `memory/`, not merely a permissive host policy.

5.2 Trusted computing base (assumed)

- Deterministic interpreter + Gate, schemas, value store / trace.

- **No LLM is in the TCB.**
- Host-supplied configuration is **axiomatic**: TrustedQuery and its label, capability grants, tool catalog, Q-LLM schemas, and DeclassPolicy are assumed correct, not attested by the kernel.

5.3 Out of scope / non-goals (selected)

Aligned with project security policy and README:

- Supply-chain compromise, side channels, DoS beyond RunLimits, compromised host.
- Conformance as safety; allow-all declassifiers.
- Typed enforcement that DeclassPolicy never consults a model (determinism is required of the declassifier as discipline, not enforced by the type system).
- Atomicity / rollback of already-committed effects.
- Python sandboxing, non-interference proofs, or provider confidentiality (quarantine does not hide data from the model provider the host chose).
- Text-to-text distortions with no unauthorized side effect (e.g. a wrong summary shown only to the user), except insofar as provenance can help UI disclosure—consistent with CaMeL’s stated non-goals for some text-only attacks [Debenedetti2025Camel].

5.4 Worked demo (fixture, not production mail)

The public demo/email_exfil.py (and its tests) exercises one agent shape under a demo policy such as “recipient is user”:

1. Legitimate summarize-and-send commits under policy.
2. Injected “forward contacts to attacker” stays inert when the planner is honest (Invariant A: planner never saw the body).
3. Malicious plan that mails contacts to an attacker is blocked (Invariant B).
4. A stricter fixture: third-party contacts cannot be mailed even to the requesting user without explicit declassification.

These scenarios are **fixtures**. They are not latency or success-rate claims, and not a production email system.

6. Implementation status

Fact	Value (as of 2026-10-07)
Repository	https://github.com/gianlucamazza/reasoning-kernel
License	MIT
PyPI name	capability-reasoning-kernel (import package reasoning_kernel; the name reasoning-kernel was taken on PyPI by an unrelated project)
Version referenced	0.6.3 (Development Status :: 4 - Beta; pre-1.0 API may change until 1.0)
Language	Python 3.12+ (classifiers include 3.12–3.14)
Site card	Active research software / reference implementation on gianlucamazza.it research & projects hubs
Companion essay	https://gianlucamazza.it/en/blog/reasoning-kernel-untrusted-data (2026-10-02)
Zenodo DOI for this whitepaper	None assigned yet (do not invent; do not reuse emotional-memory DOIs)

Provider adapters (Anthropic, OpenAI, DeepSeek-compatible, plus a deterministic FakeProvider for key-free tests) exist behind one interface. Live qualification notes in the README are **release engineering receipts**, not independent security evaluations of every model variant.

7. Relation to CaMeL and prior art

CaMeL [Debenedetti2025Camel] is the primary scientific reference for the strong Dual-LLM + capabilities + policy interpreter approach, evaluated on AgentDojo [Debenedetti2024AgentDojo]. Reasoning Kernel **implements a CaMeL-like topology in Python as a small reference**, with its own module layout, conformance CLI, and packaging. This paper does **not** restate CaMeL’s AgentDojo percentages as results of Reasoning Kernel, and does not claim equivalence of interpreters or policy languages.

Dual LLM [Willison2023] supplies the conceptual split between privileged planning and quarantined parsing without tools. Reasoning Kernel follows the strong reading: both reasoners untrusted; the Gate is the commit boundary.

Adjacent practice (same author, different artifacts): other systems on the author’s site (e.g. mklang fencing of untrusted interpolations) address related “keep data off the effect path” concerns with different mechanisms. They are not evidence for Reasoning Kernel’s Gate.

Heuristic prompt defenses (spotlighting, sandwiching, instruction hierarchy training) remain complementary defense-in-depth layers for hosts; they are not substitutes for a verification boundary on effects.

8. How to cite and use the software

Until Zenodo mints a DOI for this working paper, cite the software and essay by URL and version:

Mazza, G. (2026). *reasoning-kernel* (Version 0.6.3) [Computer software]. <https://github.com/gianlucamazza/reasoning-kernel>

Mazza, G. (2026, October 2). *Untrusted data must not own control flow*. <https://gianlucamazza.it/en/blog/reasoning-kernel-untrusted-data>

After Zenodo deposit, replace the working-paper citation with the Zenodo DOI landing page. **Do not** cite emotional-memory DOIs (10.5281/zenodo.19972258, 10.5281/zenodo.22724258) as Reasoning Kernel identifiers—they belong to a different research line.

Install:

```
pip install capability-reasoning-kernel==0.6.3
```

Pin exact versions when producing conformance evidence. Read `SECURITY.md` and `docs/CONFORMANCE.md` before treating any green report as host readiness.

9. Conclusion

Reasoning Kernel is a **reference topology** for tool-using LLM agents: mediate what planners see, quarantine what parsers touch, and commit effects only through a deterministic Gate under host policy and provenance. The public claim stays within what the tree and tests support. Publishing this working paper on Zenodo creates the artifact that, on the author’s site audit rules, may later unlock ordinary content cross-references—without inventing a DOI ahead of deposit, and without conflating unrelated publications.

References

- [Debenedetti2025Camel] Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). *Defeating Prompt Injections by Design*. arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [Debenedetti2024AgentDojo] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks.
- [Willison2023] Willison, S. (2023). The Dual LLM pattern for building AI assistants that can resist prompt injection. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>

- [Mazza2026RK] Mazza, G. (2026). reasoning-kernel [Computer software]. <https://github.com/gianluca-mazza/reasoning-kernel> · <https://pypi.org/project/capability-reasoning-kernel/>
 - [Mazza2026Essay] Mazza, G. (2026, October 2). Untrusted data must not own control flow. <https://gianluca-mazza.it/en/blog/reasoning-kernel-untrusted-data>
-

Appendix A — Document provenance (non-normative)

Prepared 2026-10-07 (CEST) from public sources: GitHub README / SECURITY.md, PyPI metadata for `capability-reasoning-kernel` 0.6.3, live essay on gianluca-mazza.it, and site project card `reasoning-kernel` (MIT, reference implementation). Emotional-memory Zenodo DOIs intentionally excluded. No peer-review or publication claim is made by the existence of this draft PDF/Markdown pack.